BLOCK 41

# DataFrames, Series, and Inspecting Tabular Data

Python 101 • Day 6 • Tabular Data and Relational Thinking

**Today's shift**

record-by-record

**table-by-table**

Same Python thinking.
Higher-level data operations.

BLOCK 41

# What this block covers

30 minutes instruction + 30 minutes guided practice

1 **Load a CSV into pandas**

pd.read_csv() creates a DataFrame from a file.

2 **Understand the table model**

Rows, columns, index, DataFrame, and Series.

3 **Inspect before transforming**

Use head, tail, shape, columns, dtypes, and info.

4 **Produce an inspection report**

Describe a dataset before cleaning it.

**Mental model: pandas packages common loop patterns into table operations.**

BLOCK 41

# From CSV dictionaries to a DataFrame

Day 5 taught the mechanics; Day 6 raises the level of expression.

## Day 5: one row at a time

```
import csv

with open("data/raw/equipment.csv",
encoding="utf-8") as file:
    reader = csv.DictReader(file)

    for record in reader:
        print(record)
```

CSV row → dictionary → process one record

## Day 6: whole table

```
import pandas as pd

records = pd.read_csv(
    "data/raw/equipment.csv"
)

print(records.head())
```

CSV file → DataFrame → inspect and transform table

BLOCK 41

# Why use a DataFrame?

A dataframe lets us describe data operations instead of managing every loop by hand.

**Filter rows**

Keep only records that match a condition.

**Select columns**

Focus on fields needed for the question.

**Summarize groups**

Count, average, sum, and compare categories.

**Loop pattern students already know**

```
active_records = []

for record in records:
    if record["status"] == "active":
        active_records.append(record)
```

**Same idea with a dataframe**

```
active_records = data[
    data["status"] == "active"
]
```

BLOCK 41

# Importing pandas

Pandas is third-party code we intentionally depend on.

## The standard import

```
import pandas as pd

records = pd.read_csv(
    "data/raw/equipment.csv"
)
```

## What pd means

`pd` is the conventional alias for pandas.

`pd.read_csv(...)` means use pandas' CSV loading capability.

**Connection to Day 5: pandas is a dependency, so the project environment must have it installed.**

BLOCK 41

# What is a DataFrame?

Beginner definition: an in-memory table with named columns and rows.

| index | record_id | source | status | error_count |
|---|---|---|---|---|
| 0 | EQ-1001 | maintenance | active | 0 |
| 1 | EQ-1002 | logistics | active | 2 |
| 2 | EQ-1003 | finance | inactive | 0 |

**Columns**
record_id, source, status...

**Rows**
Each horizontal record

**Index**
Pandas row labels

BLOCK 41

# Rows, columns, and index

Do not confuse pandas row labels with business identifiers.

**DataFrame**

The whole table: many columns, many rows.

**Column**

A named vertical field such as status.

**Row**

One horizontal record in the table.

```
index       record_id
-----       ---------
0           EQ-1001
1           EQ-1002
2           EQ-1003
```

Index = pandas row label
record_id = business data field

**Later, this distinction prepares students for primary keys and joins.**

BLOCK 41

# What is a Series?

Selecting one column usually produces a pandas Series.

## DataFrame ≈ table

```
records
```

```
record_id | source | status |
error_count
EQ-1001   | maint. | active | 0
EQ-1002   | logs   | active | 2
```



## Series ≈ one labeled column

```
statuses = records["status"]

print(type(statuses))
```

```
0    active
1    active
2    inactive
```

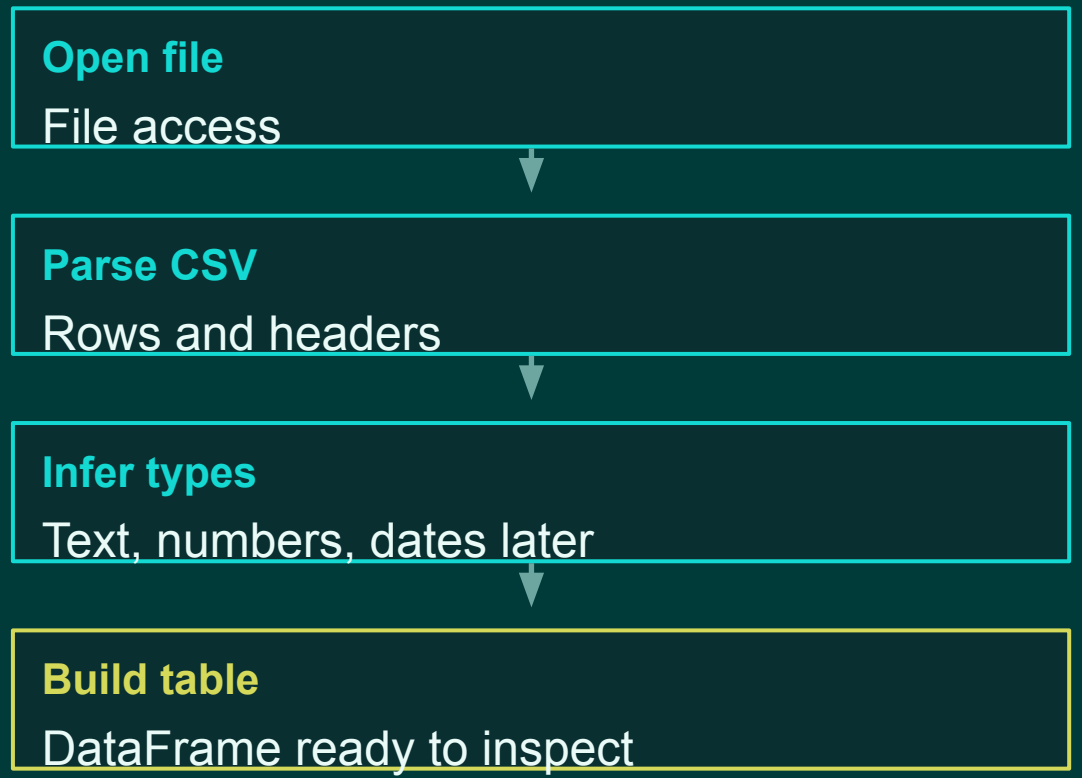
BLOCK 41

# Loading CSV data

Pandas handles several lower-level steps for us.

```
import pandas as pd

records = pd.read_csv(
    "data/raw/equipment.csv"
)
```




**Abstraction: the work still happens, but the library performs it on our behalf.**

BLOCK 41

# Inspect before processing

A professional habit: understand the table before changing it.

**Rows?**
How much data did we load?

**Columns?**
Are the expected fields present?

**Types?**
Did pandas infer useful types?

**Samples?**
Do the values look reasonable?

**Missing?**
Which fields appear incomplete?

**Do not begin transforming an unfamiliar dataframe until you inspect it.**

BLOCK 41

# First look: head() and tail()

Sample the beginning and the end of the data.

## head()

```
print(records.head())

print(records.head(10))
```

Useful for seeing column names, example values, and obvious formatting problems.

## tail()

```
print(records.tail())
```

Useful for finding footer rows, strange final records, or truncated data.

BLOCK 41

# Structural inspection

Use quick attributes to learn the size and shape of the table.

## shape

```
print(records.shape)

# (1000, 6)
```

Rows first, columns second.

## columns

```
print(records.columns)
```

Expected fields must exist before processing.

## dtypes

```
print(records.dtypes)
```

Pandas assigns a data type to each column.

BLOCK 41

# Understanding inferred data types

For now, treat object as commonly text-like in many datasets.

```
record_id        object
source           object
status           object
record_count      int64
completion        int64
error_count       int64
```

### What to ask

- Are numeric columns actually numeric?
- Are dates still being read as text?
- Does any type look suspicious?

**Inspection is not busywork. It protects every later transformation.**

BLOCK 41

# Using info()

One compact view of structure, missingness, and data types.

```
records.info()
```

## Look for

- Row count
- Column names
- Non-null counts
- Data types
- Memory summary

## The key question

Which columns contain missing values, and which columns appear complete?

**Students should understand the purpose, not memorize every line of output.**

BLOCK 41

# Common beginner gotchas

Small syntax differences matter when using pandas objects.

## Incorrect

```
records.shape()

# TypeError
```

shape is an attribute, not a method.

## Correct

```
records.shape

# (1000, 6)
```

Use parentheses only when calling a method.

BLOCK 41

# Method or attribute?

A quick mental model helps students avoid shape().

**Attribute**

A stored property of the object. Access it directly.

```
records.shape
records.columns
records.dtypes
```

**Method**

An action the object can perform. Call it with parentheses.

```
records.head()
records.tail()
records.info()
```

**Ask: am I reading a property, or asking the object to do something?**

BLOCK 41

# The Day 5 → Day 6 shift

Students should still understand that the table contains individual records.

## Day 5 mindset

```
for each record:
    normalize
    validate
    classify
    append output
```

Record-by-record processing

## Day 6 mindset

```
load table
inspect structure
select/filter columns
transform dataframe
```

Table-by-table analysis

BLOCK 41

# Instruction recap

Before practice, students should be able to explain each inspection tool.

**read_csv()**
Load a CSV into a dataframe

**head()**
Show first rows

**tail()**
Show final rows

**shape**
Rows and columns

**columns**
Column names

**dtypes**
Column data types

**info()**
Non-null counts and structure

**Now we practice the habit: load → inspect → understand → then transform.**

BLOCK 41

# Guided exercise setup

Use the unfamiliar CSV: data/raw/equipment_activity.csv

## Example columns

```
record_id
source
status
record_count
completion
error_count
processed_date
```

## Student deliverable

For each exercise, record a specific observation instead of only copying output.

**Timing: 30 minutes total guided practice**

BLOCK 41

# Exercise 1: load the file

Goal: confirm pandas import and CSV loading work.

```
import pandas as pd

records = pd.read_csv(
    "data/raw/equipment_activity.csv"
)
```

**Task**

Run the program. If it fails, identify whether the problem is an import issue or a file path issue.

**Timebox: 5 minutes**

BLOCK 41

# Exercise 2: inspect first and last rows

Goal: make observations from sample records.

```
print(records.head())
print(records.tail())
```

**Write down**

- One observation about the first rows
- One observation about the final rows
- Anything that looks suspicious

**Timebox: 6 minutes**

BLOCK 41

# Exercise 3: inspect shape

Goal: report the number of rows and columns.

```
print(records.shape)
```

**Student report**

```
Number of rows:  _________
Number of columns: _________
```

**Remember: shape is an attribute, not shape().**

BLOCK 41

# Exercise 4: inspect columns and types

Goal: identify text-like, numeric, and suspicious columns.

```
print(records.columns)
print(records.dtypes)
```

**Classify columns**

```
Text-like columns:
Numeric columns:
Suspicious inferred types:
```

**Timebox: 7 minutes**

BLOCK 41

# Exercise 5: use info()

Goal: identify missing values and complete columns.

```
records.info()
```

**Answer these questions**

- Which columns contain missing values?
- Which columns appear complete?
- Does the row count match records.shape?

**Timebox: 7 minutes**

BLOCK 41

# Applied challenge: inspection report

Load a second unfamiliar dataset. Do not clean it yet.

## Required report fields

- File name
- Number of rows
- Number of columns
- Column names
- Column data types
- Columns with missing values
- Three observations from samples

## Success criteria

The report should help another person understand the dataset before any transformation begins.

**Timebox: remaining practice time**

# Block 41 wrap-up

The key skill is not memorizing methods; it is learning how to inspect a table.

**Load**

pd.read_csv()

**Inspect**

head, tail, shape, columns, dtypes, info

**Understand**

Make observations before changing data

**Next block: selecting columns, filtering rows, sorting records, and creating calculated columns.**

Question to carry forward: What operation am I trying to perform on this table?